Altium MCP 使用说明书

编制：连子恒 | 对应提交项 5 | 初稿时间窗：2026.09.26—09.30 | 本版：v0.2.0（含 10 月增补的 EMC 审计节，一并收录备查）

通过 Model Context Protocol (MCP) 让 AI 助手连接 Altium Designer，实现原理图/PCB 数据查询、设计审查、自动化操作

文档版本：v0.2.0（2026-10-25） | 适用版本：Altium Designer 22+ | 运行环境：Windows 10/11 | Node.js 20+ | 协议：MCP stdio

一、项目概述

altium-mcp 是一个 MCP 服务器，将 Altium Designer 暴露给 AI 助手（如 Claude、OpenCode 等）。AI 助手可以通过 MCP 协议调用工具，查询 PCB/原理图数据、驱动 DelphiScript 自动化、执行设计审查。

核心能力

能力域	说明	工具数
环境配置	检测 Altium 安装、配置路径、健康检查	3
工作区查询	列出打开的项目和文档	1
原理图读取	导出元件、连线、网络、电源端口等	1
原理图编辑	放置元件、连线、网络标签、电源端口、删除、读回（15 个动作）	1
EMC 审计	原理图侧确定性 EMC 规则引擎（12 条规则，证据链输出）	1
PCB 读取	层信息、设计规则、网络、元件、引脚	7
PCB 写入	移动元件、创建网络类、控制层可见性	4
布局复制	识别同封装元件组并复制布局	2

库操作	搜索元件、导入库目录	3
输出作业	运行 OutputJob 生成制造文件	2
截图	截取 PCB/原理图编辑器视图	1
AI Skill	9 个专业设计审查与建议技能	9

设计原则：默认采用"建议模式"——AI 可以读取设计数据并给出建议，但写操作（修改原理图/PCB）被特性标志门控，默认禁用，需要手动开启。

二、快速开始

2.1 环境要求

项目	要求
操作系统	Windows 10/11
Altium Designer	22+ 推荐（X2.EXE）
Node.js	20+
MCP 客户端	任何 MCP 兼容客户端（OpenCode、Claude Desktop 等）

2.2 安装步骤

```
# 安装依赖
npm install

# 编译 TypeScript
npm run build

# 启动 MCP 服务器
npm start
```

2.3 首次使用流程

1. get_server_status → 2. configure_altium_exe → 3. altium_ping → 4.

get_workspace_projects → 5. 业务工具

1. **get_server_status**：确认工作区路径和 X2.EXE 是否被发现（同时自动同步脚本到工作区）
2. **configure_altium_exe**：如果 X2.EXE 未找到，用此工具配置路径
3. **altium_ping**：验证 DelphiScript 桥接在 Altium 内响应
4. **get_workspace_projects**：发现打开的项目和文档路径
5. **业务工具**：开始使用 PCB/原理图/库等工具

PCB 工具前置条件：使用 PCB 读取工具前，必须在 Altium 中**聚焦**目标 .PcbDoc 文档（点击使其成为活动文档）。

三、架构原理

3.1 工作流程

Node 写 request.json → 启动 X2.EXE -RScriptingSystem → DelphiScript 执行 Run → 写 response.json → Node 读取结果

3.2 关键机制

机制	说明
请求文件	Node 写 `request.json` 到工作区（默认 `%USERPROFILE%\.altium-mcp\workspace`）
脚本启动	通过 `-RScriptingSystem:RunScript` 启动 Altium 并运行 `Altium_API.PrjScr`
响应文件	DelphiScript 执行后写 `response.json`
文件锁	`.bridge.lock` 序列化并发工具调用（等待 60s，超时 120s）
错误日志	失败时写 `bridge_last_error.txt`

3.3 坐标单位

场景	单位	说明
原理图坐标	mils	`x_mils, y_mils`

PCB 读取　mils　　get_component_pins, get_selected_components

PCB 写入　mils × 10000 PCB 内部单位，set_component_position 用

四、MCP 工具详解

共 25 个工具，按功能域分组。标注 只读 写入 默认禁用 表示工具性质。

4.1 环境配置

get_server_status 只读

用途：检查 MCP + Altium 桥接配置

参数：无

返回：JSON { platform, workspaceRoot, altiumExeFound, scriptProjectFound, bundledScriptsDir, bridgeLastError, diagnosticsHint, env }

首次调用会自动同步脚本包到工作区 `AltiumScript/` 目录。

configure_altium_exe 配置写入

用途：保存 Altium X2.EXE 路径到配置文件

参数：

- `path` (string, 必需)：X2.EXE 的绝对路径

返回：{ ok, altiumExePath }

altium_ping 只读

用途：验证 DelphiScript 桥接在 Altium 内响应

参数：无

返回：{ success, result: { protocolVersion, pong } }

4.2 工作区查询

get_workspace_projects 只读

用途：列出打开的工作区条目（项目 + 文档）

参数：无

返回：JSON { projectCount, focusedProjectFullPath, projects[].{ index, projectFullPath, projectFileName, documents[].{ kind, fullPath, fileName } } }

包含非设计条目（Free Documents, AltiumScript）。

4.3 原理图读取

get_schematic_data 只读

用途：导出原理图纸数据，支持按 bucket 过滤

参数（全部可选）：

- schematic_full_path：.SchDoc 绝对路径（优先）
- project_full_path：.PrjPcb/.PrjScr 绝对路径
- schematic_sheet_file_name：文件名（如 Sheet1.SchDoc）
- include_queries：string[] 过滤器，支持：sheet | components | wires | buses | net_labels | power_ports | text_labels | junctions | ports | off_sheet_connectors | sheet_symbols | directives | figures | harness | drawing_objects | all。省略 = 默认 { components, drawing_objects }

返回：按请求 bucket 分组的 JSON 数组，坐标单位 mils

定位优先级：schematic_full_path > project_full_path + sheet > 聚焦项目

4.4 原理图编辑

edit_schematic 写入 默认禁用

用途：基于 action 的单工具原理图编辑

启用： 需设置 `ENABLE_SCHEMATIC_WRITES = true`

参数：

- `action`（必需）：动作类型
- `schematic_full_path` 或 `project_full_path + schematic_sheet_file_name`

支持的 15 个动作：

Action	说明	关键参数
set_component_transform	移动/旋转元件	designator, x_mils?, y_mils?, rotation_deg? (0/90/180/270)
set_component_parameters	设置元件参数	designator, parameter_names[], parameter_values[]
place_component	放置元件	lib_reference, designator, x_mils, y_mils, rotation_deg?, sch_library_path
add_text	添加文本	text, x_mils, y_mils
add_net_label	添加网络标签	net_name, x_mils, y_mils, rotation_deg?
add_wire	添加导线	wire_points_csv（如 "0,0,1000,0,1000,500"）
place_wire	add_wire 别名	同 add_wire
place_net_label	add_net_label 别名	同 add_net_label
add_bus	添加总线	同 add_wire
add_bus_entry	添加总线入口	wire_points_csv（4 个数）
place_power_port	放置电源端口	power_port_style, x_mils, y_mils, net_name?, rotation_deg?, show_net_name?
place_gnd	放置 GND	x_mils, y_mils（默认 style=gnd_power, net=GND）

place_vcc	放置 VCC	x_mils, y_mils (默认 style=arrow, net=VCC)
delete_object	按类型删除对象	object_type (wire/bus/net_label/power_port/compone 等)；定位：designator（元件）/ x_mils+y_mils（±25 点选）/ 矩形区域 / delete_all；返回 deleted_count AI 核对
get_component_info	读元件详情	designator；返回参数与引脚电气热点坐标（布线端点必 确等于热点坐标）

不自动保存。操作后需用户手动在 Altium 保存。建议操作后用 get_schematic_data 验证。

4.5 PCB 读取

所有 PCB 读取工具要求用户先在 Altium 中**聚焦** .PcbDoc 文档。

get_pcb_layers 只读

用途：列出电气/机械层元数据

返回：层列表，含名称、类型、可见性

get_pcb_rules 只读

用途：列出 PCB 设计规则

返回：规则描述（名称、类型、作用域）

get_all_nets 只读

用途：列出 PCB 上所有网络名

返回：String[] 网络名

get_pcb_layer_stackup 只读

用途：物理层叠结构（介电层、厚度）

返回：层叠结构 JSON

get_all_designators 只读

用途：PCB 上所有元件参考位号

返回：String[] designators

get_component_pins 只读

用途：指定元件的引脚级数据

参数：designators (string[], 必需, 至少 1 个)

返回：每个 designator 的引脚信息（pinNumber, netName, padX, padY, layer）

get_selected_components 只读

用途：获取当前选中 PCB 元件的坐标

返回：JSON[] { designator, x, y, rotation }，无选择则空

用户需先在 Altium PCB 编辑器中选择元件。

4.6 PCB 写入

所有 PCB 写入工具默认禁用，需设置 ENABLE_PCB_WRITES = true 并重新编译。

set_component_position 写入

用途：移动单个 PCB 元件到绝对位置

参数：designator, x (mils×10000), y (mils×10000), rotation? (度)

move_components 写入

用途：批量相对移动 PCB 元件

参数：designators[], x_offset (mils), y_offset (mils), rotation? (度, 默认 0)

create_net_class 写入

用途：创建网络类并添加网络（已存在则只加新成员）

参数：class_name, net_names[]

不存在的网络名会被静默跳过。

set_pcb_layer_visibility 写入

用途：切换 PCB 层可见性

参数：layer_names[]（显示名），visible（bool）

4.7 布局复制

layout_duplicator 只读

用途：查找相同封装的元件组，用于布局复制

返回：元件组列表，含源/目标候选

layout_duplicator_apply 写入 默认禁用

用途：从源元件复制布局几何（走线/圆弧/填充/过孔）到目标元件

参数：source_designators[]，destination_designators[]

要求源/目标封装匹配。

4.8 库操作

get_library_symbol_reference 只读

用途：获取项目元件的库符号引用

返回：JSON[] { lib_reference，footprint，description，library_path }

search_library_symbol 只读

用途：按名称搜索原理图符号

参数：

- symbol_name（必需）：符号名
- library_path（可选）：指定 .SchLib 路径，省略则搜全部已安装库

空 symbol_name + 指定 library_path 可枚举 IntLib 中所有元件。
import_library_components 推荐

用途：枚举项目目录 .SchLib 中所有元件并缓存目录到记忆

参数：library_path (string, 必需)：.SchLib 绝对路径（仅项目目录）

返回：{ success, library_name, component_count, project_name, memory_files, sample_components[] }

关键：导入后优先读 memory/short-term/{project}/library-catalog.md 来解析 lib_reference + sch_library_path，避免重复查 Altium。
仅支持项目目录下的 .SchLib，**不要**指向 AD 默认库目录。

4.9 输出作业

get_output_job_containers 只读

用途：列出 .OutJob 文件中的输出容器

返回：JSON[] { name, outputType, enabled }

run_output_jobs 生成文件

用途：执行输出作业容器，生成制造文件

参数：container_names (string[], 必需)

返回：{ success, output_file_paths }

4.10 截图工具

take_view_screenshot 只读

用途：截取 PCB 或原理图编辑器视图为图片

参数：view_type ("pcb" | "sch", 可选, 默认 "pcb")

返回：{ screenshot_file_path }

file_mode_capabilities 只读

用途：报告离线文件模式能力（仅 Node，无 Altium 进程）

返回：能力标志 JSON

create_schematic_symbol 写入 默认禁用

用途：在 SchLib 中创建新原理图符号

启用：需设置 `ENABLE_SYMBOL_CREATION = true`

参数：symbol_name, pins[] ("number,name,type"), part_count?, description?

4.11 EMC 审计（v0.2.0 新增）

emc_audit 分析

用途：对编译后网表与元件参数执行**确定性** EMC 规则检查（共模滤波、去耦、接地、开关噪声、接口防护），输出带证据链的审计结果——每条 finding 附命中网络/元件证据，通过项计入 checks_passed 审计轨迹，可进评审存档。

参数：与 edit_schematic 相同的页面定位（`schematic_full_path` 或 `project_full_path` + `schematic_sheet_file_name`）；依赖 `fetchSchematicAuditData` 携带的元件参数。

返回：findings[]（rule_id、severity、证据）、checks_passed[]（rule_id、summary）、统计汇总。

12 条规则：

规则 ID	类别	默认级别	检查内容
EMC-S001	传导	warning	AC 输入侧未识别到共模电感（FL 前缀或 Value/Description 含 common mode/CM choke/共模）
EMC-S002	传导	warning	L-N 之间未识别到 X 电容
EMC-S003	传导	info	X 电容容值超出常规范围（默认阈值 2.2uF，可配置）

EMC-S004 传导 warning 存在 PE 网络但未识别到 Y 电容

EMC-S005 传导 warning Y 电容容值偏大（默认 4.7nF，可配置；受漏电流限值约束）

EMC-S010 去耦 warning 电源轨上无去耦电容（每个含 IC 引脚的电源轨）

EMC-S011 去耦 info 电源轨缺少高频小电容（仅有 ≥1uF、无 ≤0.47uF）

EMC-S020 接地 info 多接地系统（PGND/SGND 等 ≥2 个地网络）未识别到 0Ω/磁珠/Y 电容桥接

EMC-S030 开关 info 变压器/功率电感绕组未识别到 R-C/RCD 吸收网络

EMC-S031 开关 info MOSFET 栅极网络上无串联电阻

EMC-S032 接口 warning 晶振两脚未识别到对地负载电容

EMC-S040 接口 info 连接器信号网络直达 IC 且无 TVS/串联电阻/磁珠防护

阈值与网络识别可配置：X/Y 电容阈值、小电容阈值、网络名识别模式（NET_PATTERNS）均按公司规范在配置中调整；规则依据为通用 EMC 设计实践（Ott 等参考文献），不含虚构条款编号，正式引用条款以公司标准为准（阈值终值待 EMC 工程师确认）。

五、AI Skill 技能

项目内置 9 个专业设计 Skill，是面向 AI 助手的提示词模板，覆盖从电路设计到文档生成的完整流程。Skill 标记。

5.1 电路拓扑建议 Skill

核心功能：根据用户描述的电路意图，推荐电路拓扑结构、计算元件值、解释设计权衡

输入：电路功能需求、性能指标（如输入/输出电压、电流、效率要求）

输出：结构化的拓扑方案，含推荐电路图、元件参数、设计要点

使用场景：

- 设计新电路功能时，需要选择合适的拓扑结构
- 要求计算元件参数（电阻、电容、电感值）

- 对比不同拓扑的优劣

典型对话："我需要一个 5V 转 3.3V 的电源，最大电流 2A，效率不低于 90%"

5.2 元件选型与连接方案 Skill

核心功能：根据电路需求匹配推荐元件并生成选型报告与连接方案

输入：用户需求描述 + 记忆中的库元件信息（来自 import_library_components）

输出：元件选型报告 + 连接方案

使用场景：

- 描述电路功能需求，要求元件选型
- 生成元件连接方案
- 结合库目录记忆推荐合适的元件

典型对话："帮我选一个低功耗运放，用于电池供电的传感器信号放大"

5.3 原理图审查审计 Skill

核心功能：读取 Altium 原理图数据，分析元件连接关系，生成审查审计报告

输入：原理图数据（get_schematic_data 结果）+ 长期记忆

输出：结构化审计报告，含问题清单、风险点、改进建议

使用场景：

- 审查原理图连接关系正确性
- 检查设计规范遵守情况
- 审计设计完整性和潜在风险

5.4 设计规则推导 Skill

核心功能：根据电路意图生成 DRC 设计规则配置建议

输入：电路类型、性能指标、工艺要求

输出：设计规则建议报告（线宽、线距、过孔、阻焊、铺铜规则）

使用场景：

- 设置/检查 DRC 规则
- 确定线宽线距要求
- 定义过孔/阻焊/铺铜规则

典型对话："这个板子有 USB 3.0 差分对，帮我推导需要的 DRC 规则"

5.5 布局策略建议 Skill

核心功能：根据电路拓扑建议 PCB 布局策略

输入：PCB 元件数据 + 原理图数据

输出：布局策略报告（功能分区、元件摆放建议、关键路径）

使用场景：

- PCB 布局规划
- 元件摆放建议
- 功能分区规划

5.6 信号完整性建议 Skill

核心功能：根据 PCB 层叠结构和信号类型推导线宽、线距、阻抗控制规则

输入：PCB 层叠结构、设计规则、信号类型

输出：信号完整性设计指导报告

使用场景：

- 阻抗匹配分析
- 差分对走线规则
- 高速信号完整性评估

典型对话："4 层板，TOP-GND-PWR-BOTTOM，50Ω 单端阻抗怎么走？"

5.7 PCB 设计审查顾问 Skill

核心功能：读取 PCB 设计数据执行综合审查，含原理图-PCB 一致性检查

输入：PCB 数据 + 原理图数据

输出：结构化综合审查报告

使用场景：

- 设计审查（DRC 之外的设计合理性）
- 原理图-PCB 一致性检查
- 网络命名规范检查

5.8 BOM 分析与优化 Skill

核心功能：审查 BOM 物料清单，分析单一来源、生命周期、替代料

输入：原理图中的元件数据

输出：BOM 分析报告（风险元件、替代料建议、采购优化）

使用场景：

- 分析 BOM 物料风险
- 检查单一来源零件
- 元件生命周期状态评估
- 优化采购成本

5.9 设计文档生成器 Skill

核心功能：分析设计数据，自动生成电路功能说明、设计决策记录、评审文档

输入：设计数据 + 长期记忆

输出：结构化文档（设计说明书、决策记录、评审材料）

使用场景：

- 生成设计说明书

- 记录设计决策
- 准备评审材料
- 输出技术文档

六、典型使用场景

场景 1：新项目启动检查

```
1. get_server_status      // 检查环境
2. altium_ping            // 验证桥接
3. get_workspace_projects // 列出项目
4. get_schematic_data     // 读取原理图
   include_queries: ["components", "net_labels", "power_ports"]
5. get_pcb_layers         // 读取 PCB 层（需先聚焦 PcbDoc）
6. get_all_designators    // 列出元件
```

场景 2：原理图审查

```
1. get_schematic_data
   schematic_full_path: "C:\\项目\\Sheet1.SchDoc"
   include_queries: ["all"]
2. 调用 skill-schematic-audit 进行审查
3. 调用 skill-bom-analyzer 分析物料
4. 生成审查报告
```

场景 3：PCB 设计审查

```
1. 聚焦目标 .PcbDoc
2. get_pcb_layer_stackup  // 层叠结构
3. get_pcb_rules          // 设计规则
4. get_all_nets           // 网络列表
5. get_all_designators    // 元件列表
6. get_component_pins     // 关键元件引脚
   designators: ["U1", "U2", "J1"]
7. 调用 skill-design-review-advisor 综合审查
8. 调用 skill-signal-integrity-advisor 高速信号分析
```

场景 4：元件选型与放置

```
1. import_library_components  // 导入库目录
   library_path: "C:\\项目\\libs\\my.SchLib"
2. 读取 memory/short-term/{project}/library-catalog.md
3. 调用 skill-component-selection-prompt 选型
4. （启用写操作后）edit_schematic place_component 放置元件
```

场景 5：布局复制

```
1. 聚焦 .PcbDoc
2. layout_duplicator  // 查找同封装元件组
3. 确认源/目标映射
4. （启用写操作后）layout_duplicator_apply 复制布局
```

场景 6：生成制造文件

```
1. get_output_job_containers  // 列出 OutJob 容器
2. run_output_jobs             // 执行
   container_names: ["Gerber Files", "Drill Files", "BOM"]
```

七、错误处理

错误	原因	解决方法
AD_NOT_FOUND	X2.EXE 路径错误	调用 configure_altium_exe 设置正确路径
SCRIPT_PROJECT_NOT_FOUND	脚本包未同步	调用 get_server_status（自动同步）
TIMEOUT	脚本挂起或断点	清除 Altium IDE 断点；删除 .bridge.lo
LockTimeout	上次崩溃留下锁	删除工作区目录下的 .bridge.lock
空数据	PCB 未聚焦	在 Altium 中聚焦 .PcbDoc
INVALID_BRIDGE_RESPONSE	JSON 解析失败	检查 bridge_last_error.txt
SHEET_REQUIRED / SCHEMATIC_SHEET_NOT_FOUND	原理图页面定位失败	补 schematic_full_path，或先 get_workspace_projects 确认项目已打开
COMPONENT_NOT_FOUND	位号不存在	先 get_schematic_data 读回实际位号（后 AD 可能自动改号）
PLACED_COMPONENT_NOT_FOUND /	库未安装或路	确认 sch_library_path 指向的 .SchLib

PLACE_SCH_COMPONENT_FAILED	径不可见	AD 中安装

脚本编译错误不会经过桥接通道：若 DelphiScript 编译失败（消息面板报错但 MCP 侧只看到超时/无响应），把 Altium 消息面板的错误文本粘贴到工作区 `bridge_user_reported_error.txt`，再调用 `get_server_status`——AI 即可读取（bridgeUserReportedError）并定位修复。

锁文件位置：默认在工作区目录 `%USERPROFILE%\.altium-mcp\workspace\.bridge.lock`。删除此文件可解除死锁；服务端会自动清理指向不存在进程的陈旧锁。

八、记忆协议

为减少工具调用和上下文消耗，系统维护两级持久化记忆。

8.1 短期记忆（short-term）

项目	说明
位置	memory/short-term/{项目名}/{文件名}.md
保存时机	首次查询后立即写入
加载时机	会话开始时自动加载
生命周期	当前项目/文件的整个会话期间

8.2 长期记忆（long-term）

项目	说明
位置	memory/long-term/{项目名}.md
保存时机	用户主动要求保存设计知识时
加载时机	进入项目时自动加载
生命周期	跨会话持久保留

8.3 记忆格式

```
## MEM:PCB_COMP 2026-01-15T10:35
@U1 F:QFP-48 X:3500 Y:2100 R:0
@R1 F:0603 X:1500 Y:800 R:90

## MEM:SCH_COMP 2026-01-15T10:30
@U1 L:STM32F407 X:5000 Y:3000 P:Package=LQFP48

## MEM:PIN_NET 2026-01-15T10:36
@U1.1=VCC @U1.2=GND @U1.3=GPIO_A0

## MEM:SCH_NET 2026-01-15T10:31
@VCC: [U1.1, R1.1, C1.1]
@GND: [U1.2, R1.2, C1.2]

## MEM:LIB_CATALOG 2026-01-15T10:40
@LIB test1.SchLib P:C:\...\test1.SchLib N:42
```

8.4 记忆控制指令

指令	含义
″保存记忆″	将当前短期记忆写入文件
″加载记忆″	从文件恢复记忆
″保存为长期记忆″	保存项目设计知识
″加载长期记忆″	加载项目设计知识
″清除短期记忆″	删除当前项目短期记忆文件（保留 README.md）
″重置长期记忆″	清除项目长期记忆（保留 README.md）

九、特性标志

写操作通过特性标志门控，**默认全部禁用**。开启方法：在 `src/index.ts` 和 `Altium_API.pas` 中将标志设为 `true`，然后重新编译。

标志	控制的工具	默
ENABLE_SCHEMATIC_WRITES	edit_schematic（11 个 action）	fa

ENABLE_PCB_WRITES	set_component_position, move_components, create_net_class, set_pcb_layer_visibility, layout_duplicator_apply	fa
ENABLE_SYMBOL_CREATION	create_schematic_symbol	fa

安全提示：启用写操作后，AI 助手可以修改你的设计文件。建议在启用前：

- 备份当前项目
- 在测试项目上先验证
- 密切关注 AI 的每一步写操作

启用步骤

```
// 1. 编辑 src/index.ts
export const ENABLE_SCHEMATIC_WRITES = true;
export const ENABLE_PCB_WRITES = true;
export const ENABLE_SYMBOL_CREATION = true;

// 2. 编辑 altium-scripts/Altium_API.pas
//    找到对应标志，设为 True

// 3. 重新编译
npm run build

// 4. 重启 MCP 服务器
npm start
```